Day 6 Student Notes: Claude for Linux Administration, Part 1

> **Today in one line:** Claude helps you learn Linux commands and read logs faster. **You verify before you run anything.**

What is a Linux server?

A computer with no screen that runs the website. You control it by typing commands. Everything it does is written to a **log**, its diary. When something breaks, the answer is usually in a log.

Five commands that only look

Command	What it does	Example
ls	List files in a folder	ls
cat	Show a whole file	cat df.txt
head / tail	First or last lines	tail -5 app.log
grep	Find lines with a word	grep ERROR app.log
wc -l	Count lines	wc -l app.log

A command chain

```
grep ERROR app.log | cut -d' ' -f4- | sort | uniq -c | sort -rn | head -3
```

Part	What it does
grep ERROR app.log	Keep only lines with ERROR
cut -d' ' -f4-	Drop the date and time, keep the message
sort	Put equal lines next to each other
uniq -c	Count how many times each repeats
sort -rn	Biggest count first
head -3	Show the top 3

The | (pipe) sends the output of one command into the next.

Three ways to use Claude Code with Linux

Way	Example prompt	Risk
1. Explain	"What does this command do?"	Lowest: Claude only talks
2. Investigate	"Read these logs. What is the cause?"	Low: you check the proof
3. Write	"Write a script that does this."	Medium: review it before running

Today you run the commands yourself in the VS Code terminal.

Setup

1. Unzip **server-triage.zip**. In VS Code: File, Open Folder, choose server-triage, Trust.
2. Open the Claude panel (Spark icon). Set the mode to **Plan**.
3. Open the terminal: Ctrl + ` (Mac: Cmd + `). **Windows:** click the arrow next to + and choose **Git Bash** (PowerShell has no grep).

No Git Bash? Skip the terminal steps and ask Claude to explain the commands and output.

Hands-on (prompts also in Day06_Prompts_To_Paste.txt)

1. Run it, then ask Claude to explain it

```
ls
wc -l app.log
tail -5 app.log
grep ERROR app.log | head -5
```

```
Explain this command in simple words, part by part.
Do not run it.

grep ERROR app.log | cut -d' ' -f4- | sort | uniq -c | sort -rn | head -3
```

2. Find the root cause

```
Role: You are a careful Linux troubleshooter.
Context: A made-up web server started returning errors at 14:20. The files are
app.log and syslog.txt in this folder.
Task: Find the most likely root cause. Then give the next command I should run to
check it.
Constraints: Use only what is in the files. Say "unknown" if the logs are silent.
Do not run anything.
Format: Root cause in 2 lines, then 3 evidence lines (file and what it shows), then
the next command.
```

Check: find each line Claude quotes in the files.

3. Write a safe script

```
Role: You are a Linux engineer who writes simple, safe scripts.
Context: app.log is a made-up server log. Each line starts with a date and time,
then the level (INFO, WARN, ERROR), then the message.
Task: Write log_summary.sh. It takes a log file name and prints: the number of
lines, the number of ERROR lines, and the 3 most common ERROR messages.
Constraints: Read-only commands only (grep, cut, sort, uniq, wc, head). Do not
delete or change any file. Add a short comment above each line.
Format: One script file. Then explain each line in one short sentence.
```

Read the plan, check the diff, accept. Then run **bash log_summary.sh app.log**. Then ask: *What could go wrong if I run log_summary.sh on production? Is anything in it unsafe?*

4. Quick triage

```
Role: You are a careful Linux troubleshooter.
Context: Same server. The saved checks are df.txt, free.txt and top.txt.
Task: Is CPU, memory or disk the problem? Give one line for each, then the top
priority.
Constraints: Use only these files. Read only. Do not run anything.
Format: Three lines (CPU, Memory, Disk), then "Top priority:" and one sentence.
```

5. Review a dangerous script

```
Role: You are a cautious senior Linux engineer.
Context: cleanup.sh was written by a teammate to free disk space on a production
web server. Do not run it.
Task: Review it line by line. Rate each command Safe, Risky or Dangerous and
explain why in simple words.
Constraints: Read only. Do not run or edit the script.
Format: A table: line, command, rating, what could go wrong. Then a safer way to
free the space.
```

Never run cleanup.sh. Then ask: *Give me a safer, step by step way to free disk space on this server. Start with read-only checks. Tell me what to back up first.*

Triage: three questions

	Command	Bad sign
Disk	df -h	Use% at 100
Memory	free -h	Swap used, "Out of memory"
CPU	top	Load very high

Verify before you run

1. **Understand it.** Ask Claude to explain.
2. **Look for danger words.**

3. **Dry run.** Echo it first.
4. **Test machine first.** Keep a way back (backup or snapshot).

Danger word	Why
rm -rf	Deletes forever, no Trash
chmod -R 777	Gives everyone full access
kill -9	Stops a process with no cleanup
sudo	Runs as admin

Key words

Word	Meaning
Server	A computer that runs a service, often with no screen
Log	The diary of what a system did
Root cause	The real reason the problem started
Hypothesis	A good guess that still needs proof
Pipe |	Sends one command's output into the next
Triage	Checking the biggest problems first

Check yourself

1. What does grep do?
2. What does | do?
3. Name the three ways to use Claude Code with Linux.
4. Name two danger words.
5. Claude says a command is safe. What do you still do?

Answers

1. Finds lines that contain a word.
2. Sends one command's output into the next.
3. Explain, investigate, write.
4. Any two of rm -rf, chmod -R 777, kill -9, sudo.
5. Understand it, check for danger words, dry run, test machine first.

Take-home task (20 to 30 minutes)

1. Open server-triage again. Ask Claude to find the second-biggest problem in app.log and explain it.
2. Bring one command Claude suggests that you would NOT run on production, and say why.

> **Next class, Day 7:** Claude for Linux Administration Part 2: services, cron jobs, firewall rules and hardening.